



Université Ibnou Zohr
École Nationale des Sciences Appliquées d'Agadir
Filière SDBDIA — Science des Données, Big Data et Intelligence Artificielle

Banque Centrale Populaire — Casablanca
Data Framework and Technology

RAPPORT DE STAGE

Stage d'initiation — Projet de Fin d'Année

**Conception d'une plateforme d'ingestion de données multisources avec Apache NiFi,
orchestration Airflow et centre de contrôle Flask**

Réalisé par : Mahdi Benhamadi
Élève ingénieur — 1ère année, filière SDBDIA
Sous la direction de : Mme Ajellal Fatima Zahra

Année universitaire 2025 / 2026

« L'école n'entend donner aucune approbation ni improbation aux opinions émises dans ce rapport : ces opinions doivent être considérées comme propres à leur auteur. »

Remerciements

Je tiens à exprimer ma gratitude envers l'ensemble des personnes qui ont contribué, de près ou de loin, au bon déroulement de ce projet au sein de la Banque Centrale Populaire.

Mes remerciements vont à mon encadrant ainsi qu'à l'équipe Data Framework and Technology pour leur disponibilité et le cadrage apporté tout au long du stage.

Je remercie également le corps professoral de l'ENSA Agadir pour la formation qui m'a permis d'aborder ce projet avec les bases nécessaires en informatique et en science des données.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage d'initiation, organisé sous la forme d'un Projet de Fin d'Année, au sein de la Banque Centrale Populaire. Le projet porte sur la conception de flux Apache NiFi automatisés pour l'ingestion de données depuis plusieurs sources : fichiers plats déposés dans un répertoire local (CSV, JSON, XML, TXT) et une base de données relationnelle MySQL via JDBC.

La solution développée intègre un mécanisme de gestion des erreurs (retry paramétrable et mise en quarantaine), une authentification centralisée via LDAP pour sécuriser les accès à NiFi, et un stockage des données sur HDFS partitionné par date, avec une table Hive alignée sur cette organisation. L'ensemble du pipeline est orchestré de bout en bout par Apache Airflow, avec notification du résultat de chaque exécution sur Discord.

Face à des contraintes de ressources sur la machine de développement (espace disque restreint, 16 Go de RAM), l'environnement Cloudera CDP initialement envisagé a été remplacé par une stack conteneurisée avec Docker Compose, plus légère et plus simple à mettre en œuvre et à déboguer. Cette architecture s'inspire des pipelines Big Data de production sans chercher à en reproduire l'exhaustivité, et reste déployable localement en une seule commande.

En fin de stage, une extension a été développée au-delà du périmètre initial : un centre de contrôle Flask centralisant l'administration de l'ensemble de la plateforme (Docker, Airflow, HDFS, Hive, LDAP, MySQL, notifications Discord, visualisation du pipeline NiFi) dans une interface web unique. Le résultat est une plateforme d'ingestion modulaire et versionnée sur GitHub, conçue autour de mécanismes de paramétrage (Parameter Contexts NiFi, Variables Airflow) pour rester réutilisable.

Mots-clés : *Apache NiFi, ingestion de données, Big Data, Hadoop HDFS, Apache Hive, Apache Airflow, Docker, LDAP, Flask, centre de contrôle.*

Sommaire

Introduction générale

Chapitre 1 — Contexte et présentation du projet

Chapitre 2 — Démarche d'apprentissage et choix technologiques

Chapitre 3 — Conception de l'architecture

Chapitre 4 — Réalisation technique du pipeline d'ingestion

Chapitre 5 — Le centre de contrôle Flask

Chapitre 6 — Difficultés rencontrées et démarche de résolution

Chapitre 7 — Résultats et bilan

Conclusion générale

Bibliographie

Annexes

Liste des figures et tableaux

Figures

Figure 1. Architecture globale de la plateforme, centre de contrôle Flask compris.

Figure 2. Organisation des Process Groups du flux NiFi.

Figure 3. Vue des Process Groups principaux dans l'interface NiFi.

Figure 4. Paramétrage générique d'un Process Group via un Parameter Context.

Figure 5. Arborescence HDFS partitionnée et table Hive correspondante.

Figure 6. Exécution du DAG Airflow et notifications Discord associées.

Figure 7. Structure applicative du centre de contrôle Flask.

Figure 8. Tableau de bord principal du centre de contrôle.

Figure 9. Modules Docker et Airflow du centre de contrôle.

Figure 10. Modules Hive et HDFS du centre de contrôle.

Figure 11. Visualisation interactive du pipeline NiFi dans le centre de contrôle.

Extra: MySQL insert, LDAP browsing, and README viewer modules.

Tableaux

Tableau 1. Périmètre fonctionnel fixé par le cahier des charges.

Tableau 2. Technologies retenues et rôle dans le projet.

Tableau 3. Fonctionnalités du centre de contrôle par module.

Tableau 4. Difficultés rencontrées et résolutions apportées.

Tableau 5. Livrables réalisés au regard du cahier des charges.

Liste des sigles et abréviations

Sigle	Signification
API	Application Programming Interface
BCP	Banque Centrale Populaire
CLI	Command Line Interface
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
ENSA	École Nationale des Sciences Appliquées
HDFS	Hadoop Distributed File System
HTML	HyperText Markup Language
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
LDAP	Lightweight Directory Access Protocol
NAR	NiFi Archive
PFA	Projet de Fin d'Année
PG	Process Group
RAM	Random Access Memory
REST	Representational State Transfer
SDBDIA	Science des Données, Big Data et Intelligence Artificielle
SQL	Structured Query Language
SVG	Scalable Vector Graphics
UI	User Interface
URL	Uniform Resource Locator
VM	Virtual Machine (machine virtuelle)
WSL	Windows Subsystem for Linux
XML	Extensible Markup Language

Introduction générale

Dans un contexte bancaire où la donnée constitue un actif stratégique, la capacité à ingérer, sécuriser et stocker efficacement des flux de données hétérogènes est un enjeu important pour les directions Data des établissements financiers. La Banque Centrale Populaire, à travers son pôle Data Framework and Technology, propose dans ce cadre un stage portant sur la conception de flux d'ingestion automatisés basés sur Apache NiFi, organisé et évalué comme un Projet de Fin d'Année.

L'objectif du stage est de concevoir une solution générique et réutilisable capable d'ingérer des données provenant de plusieurs sources — fichiers plats et une base relationnelle — tout en travaillant la sécurisation des accès et la robustesse face aux erreurs. Le cahier des charges fourni en début de stage fixe un périmètre large, incluant plusieurs moteurs de bases de données et plusieurs modes de dépôt de fichiers ; ce rapport précise, chapitre après chapitre, la part de ce périmètre effectivement couverte, ainsi qu'une extension développée au-delà de ce périmètre initial.

N'ayant, au démarrage, aucune expérience préalable avec l'écosystème NiFi/Hadoop, la démarche de travail adoptée combine trois canaux d'apprentissage — documentation officielle, tutoriels vidéo et expérimentation directe — menés en parallèle du développement plutôt qu'en amont de celui-ci. Chaque notion technique est ainsi apprise au moment où elle devient nécessaire à l'avancement du flux.

Ce rapport est structuré en sept chapitres. Le premier présente le contexte et les objectifs du projet. Le deuxième détaille la démarche d'apprentissage et les choix technologiques opérés, notamment le remplacement de l'environnement Cloudera CDP par une stack Docker. Le troisième décrit l'architecture conçue. Le quatrième détaille la réalisation technique du pipeline d'ingestion. Le cinquième présente le centre de contrôle Flask développé en fin de stage. Le sixième revient sur les difficultés rencontrées et la démarche de résolution de problèmes. Le septième dresse le bilan du projet au regard du cahier des charges et propose des perspectives d'amélioration.

Chapitre 1 — Contexte et présentation du projet

1.1 Organisme d'accueil

Le stage se déroule au sein de la Banque Centrale Populaire (BCP), à Casablanca, au sein du pôle Data Framework and Technology, rattaché au département Data Custodian, en charge des infrastructures et des cadres techniques liés à la donnée au sein de la banque. Il est réalisé sous l'encadrement de Mme Ajellal Fatima Zahra

1.2 Contexte et problématique

Les établissements bancaires manipulent quotidiennement d'importants volumes de données provenant de sources hétérogènes (fichiers plats, bases de données relationnelles, etc.). L'objectif de la BCP est de mettre en place un mécanisme d'ingestion unifié et sécurisé afin d'alimenter un futur Data Lake, espace de stockage conservant de grands volumes de données brutes dans leur format d'origine en vue d'un traitement ou d'une analyse ultérieure, tout en évitant la multiplier les scripts ad hoc difficilement maintenables.

Dans cette optique, le stage consiste à s'appuyer sur Apache NiFi, pour concevoir des Process Groups génériques et paramétrables, capables de traiter différents types de sources sans réécriture complète du pipeline.

1.3 Objectifs du projet

L'objectif général du stage, tel que défini dans le cahier des charges, est de concevoir, développer et déployer des flux Apache NiFi automatisés, génériques et réutilisables, permettant l'ingestion de données multisources, avec gestion des erreurs et stockage HDFS organisé par partitionnement temporel. Les objectifs spécifiques sont les suivants :

- Concevoir une architecture de flux NiFi générique et paramétrable, capable de traiter plusieurs types de sources sans réécriture complète du flux ;
- Développer l'ingestion de fichiers plats (CSV, TXT, XML, JSON) déposés dans un répertoire dédié ;
- Développer l'ingestion d'une base de données relationnelle via JDBC ;
- Sécuriser les accès à NiFi via une authentification centralisée ;
- Mettre en place une gestion des erreurs robuste (détection, isolation, rejeu) ;
- Organiser le stockage sur HDFS selon un schéma de partitionnement par date ;
- Documenter la solution et produire les livrables permettant sa prise en main par l'équipe.

1.4 Périmètre visé par le cahier des charges

Type de source	Formats / Protocoles envisagés	Mode d'ingestion envisagé
Fichiers plats	CSV, TXT délimité, largeur fixe, XML, JSON	Dépôt SFTP ou répertoire partagé, détection automatique
Bases relationnelles	Oracle, SQL Server, PostgreSQL, MySQL (JDBC)	Extraction incrémentale (colonne de suivi)
Évolutivité	Autres formats à intégrer ultérieurement (Parquet, Avro...)	Architecture conçue pour être extensible sans refonte majeure

Le tableau ci-dessous synthétise le périmètre initialement fixé par le cahier des charges. La part réellement couverte pendant le stage est détaillée aux chapitres 4 et 5, et synthétisée au chapitre 7.

Tableau 1. Périmètre fonctionnel fixé par le cahier des charges.

Chapitre 2 — Démarche d'apprentissage et choix technologiques

2.1 Un écosystème découvert en cours de stage

Au démarrage du stage, aucune expérience préalable avec Apache NiFi, Hadoop/HDFS, Hive ou Airflow n'était acquise. En revanche, une pratique antérieure du scripting Python et une bonne familiarité avec le terminal Linux ont constitué un socle utile, facilitant la prise en main des outils en ligne de commande, de Docker et des scripts d'orchestration.

La montée en compétence s'appuie sur trois canaux complémentaires : la documentation officielle des outils (NiFi, Hadoop, Airflow, Flask — voir Bibliographie), plus d'une centaine de tutoriels vidéo consacrés aux technologies utilisées, et un travail itératif d'expérimentation directe dans NiFi, complété par des échanges avec des assistants conversationnels pour déboguer des configurations et clarifier des concepts.

Cette montée en compétence est menée en parallèle du développement, chaque nouvelle notion (Process Groups, Parameter Contexts, Record API, JDBC, HDFS, sécurité, développement Flask) étant apprise juste avant d'être mise en application sur le flux ou le module en cours de construction.

2.2 Du Cloudera CDP à une stack conteneurisée

Le cahier des charges initial mentionne un déploiement sur environnement Cloudera CDP. En pratique, la machine de développement disponible pour le stage présente des ressources limitées : espace disque presque saturé et 16 Go de RAM seulement, insuffisants pour faire tourner confortablement une machine virtuelle Cloudera complète en plus des autres outils nécessaires. Une réorientation technique est donc opérée : remplacer Cloudera par une stack Docker Compose reproduisant les composants Big Data nécessaires (NiFi, Hadoop HDFS, Hive, MySQL) et par Apache Airflow pour l'orchestration.

Ce choix est motivé par plusieurs facteurs :

- **Légereté** — des conteneurs Docker consomment nettement moins de ressources qu'une VM complète, ce qui les rend compatibles avec un poste de développement aux capacités limitées.
- **Simplicité d'installation et de dépannage** — les conteneurs Docker s'appuient sur des images Linux, exécutées via WSL, ce qui facilite l'installation, la lecture des journaux et le débogage par rapport à une installation native de Hadoop, en particulier sous Windows.
- **Reproductibilité** — l'environnement démarre en une seule commande (`docker compose up -d`) et est identique sur toute machine disposant de Docker, ce qui facilite la prise en main par l'équipe d'accueil.
- **Fidélité à l'esprit du cahier des charges** — Cloudera n'étant qu'une couche d'orchestration autour de composants Hadoop standards, la stack Docker conserve les mêmes briques fonctionnelles (HDFS, Hive) sans dénaturer les objectifs du projet.

Apache Airflow est par ailleurs introduit comme couche d'orchestration, un besoin identifié dans le cahier des charges mais non explicitement rattaché à un outil précis. Airflow est retenu pour démarrer les services, déclencher le Process Group NiFi via son API REST, et notifier le résultat de chaque exécution.

2.3 Technologies retenues

Tableau 2. Technologies retenues et rôle dans le projet.

Technologie	Rôle dans le projet
Apache NiFi	Moteur d'ingestion : routage, conversion, gestion des erreurs
Apache Airflow	Orchestration : démarrage des services, déclenchement du pipeline NiFi, notifications
Hadoop HDFS	Stockage distribué partitionné (namenode/datanode conteneurisés)
Apache Hive	Couche table SQL sur HDFS, alignée avec le partitionnement
MySQL	Source relationnelle pour l'ingestion JDBC, et métastore Hive (schéma dédié)
OpenLDAP	Authentification centralisée des utilisateurs NiFi
Docker / Docker Compose	Conteneurisation et orchestration de l'ensemble des services
Discord (webhook)	Notifications d'exécution du pipeline (succès/échec)
Flask / Python	Centre de contrôle web développé en fin de stage (chapitre 5)

Chapitre 3 — Conception de l'architecture

3.1 Vue d'ensemble

L'architecture retenue s'articule autour de quatre grands blocs : l'ingestion (fichiers plats et JDBC), la conversion vers un format pivot unique, le stockage partitionné sur HDFS, et l'orchestration assurée par Airflow. L'authentification est déléguée à un annuaire LDAP, utilisé par NiFi pour sécuriser l'accès des utilisateurs. Le centre de contrôle Flask, décrit au chapitre 5, vient se superposer à cette architecture comme couche d'administration transverse.

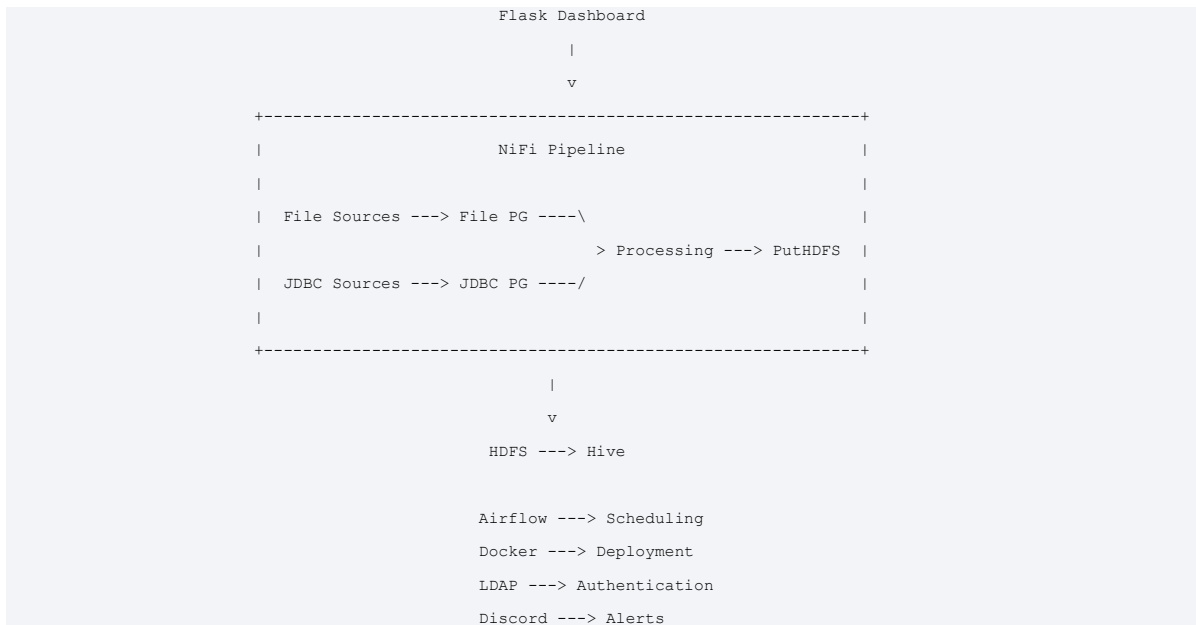


Figure 1. Architecture globale de la plateforme, centre de contrôle Flask compris.

Les différentes sources — fichiers plats ou base de données — convergent après conversion vers un format CSV unifié, avant d'être écrites sur HDFS. Ce choix d'un format pivot unique simplifie la couche de stockage aval : quelle que soit la source d'origine, le traitement de stockage est identique.

3.2 Architecture des flux NiFi

Le flux NiFi est organisé en Process Groups modulaires, reflétant une séparation claire des responsabilités :

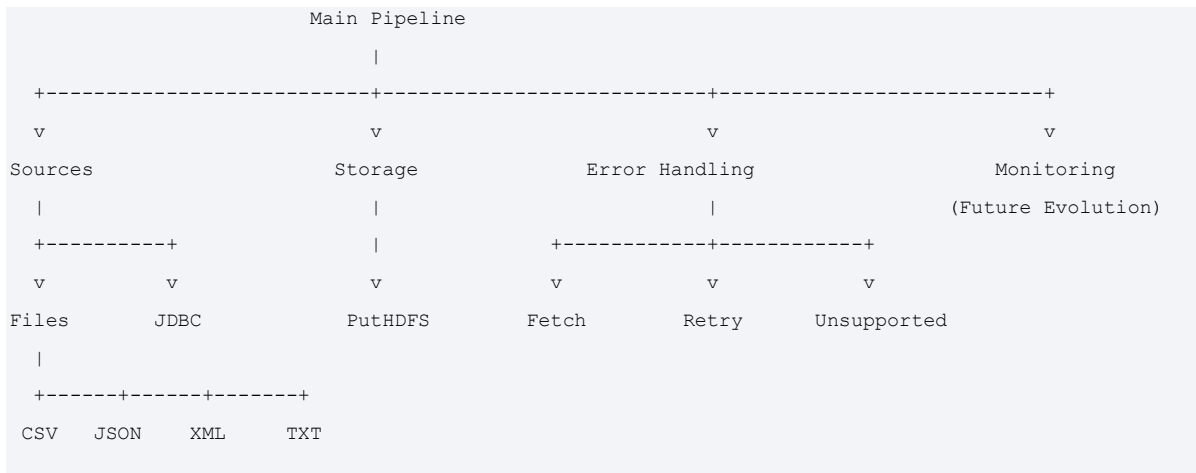


Figure 2. Organisation des Process Groups du flux NiFi.

Cette architecture en Process Groups connectés par des ports d'entrée/sortie permet d'ajouter une nouvelle source de fichiers sans modifier les Process Groups existants : il suffit de brancher un nouveau flux de conversion vers le port d'entrée du bloc Storage.

3.3 Généricité : Parameter Contexts et Variables Airflow

Pour que le flux reste réutilisable au-delà des sources initialement traitées, deux mécanismes de paramétrage sont mobilisés plutôt que de coder des valeurs en dur dans les flux :

- **Parameter Contexts (NiFi)** — les chemins, noms de connexions et autres valeurs configurables des Process Groups sont injectés via des Parameter Contexts, ce qui permet de faire évoluer une source sans modifier la logique du flux elle-même.
- **Variables (Airflow)** — les identifiants NiFi et l'URL du webhook Discord sont stockés dans les Variables Airflow plutôt qu'en dur dans le code du DAG, ce qui évite toute donnée sensible versionnée dans le dépôt Git et facilite le changement d'environnement.

3.4 Convention de partitionnement HDFS

Les données sont stockées sur HDFS selon une arborescence partitionnée par date d'ingestion :

```
/data / <source> / <entité> / year=2026 / month=07 / day=20 /
```

Le chemin est généré dynamiquement via le langage d'expression de NiFi, à partir d'attributs de métadonnées (source, entité, date d'ingestion) enrichis en amont dans le flux. Une table Hive est créée en cohérence avec cette arborescence, afin de pouvoir interroger les données en SQL sans dupliquer le stockage.

3.5 Orchestration Airflow

L'orchestration repose sur un DAG Airflow (final_ingestion_pipeline.py) découpé en modules utilitaires réutilisables plutôt qu'un script monolithique :

- `docker_utils.py` — vérification et démarrage des conteneurs Docker requis ;
- `wait_utils.py` — attente active de la disponibilité des services (readiness checks) ;
- `nifi_utils.py` — authentification et pilotage du Process Group NiFi via l'API REST ;
- `notify_utils.py` — envoi de notifications Discord (succès/échec, durée d'exécution).

Le rôle de Airflow est de coordonner les dépendances et les conditions d'attente entre les différents services (démarrage des conteneurs, attente de leur disponibilité, déclenchement du pipeline NiFi, puis envoi des notifications). Cette orchestration est complémentaire de celle de NiFi : Airflow pilote le déroulement global du workflow, tandis que NiFi gère l'exécution et l'enchaînement des traitements de données au sein du pipeline.

3.6 Sécurité et authentification

L'authentification de NiFi est migrée du mode single-user initial vers une authentification LDAP via un annuaire OpenLDAP dédié, administré via phpLDAPadmin. Cette évolution sécurise l'accès à l'interface NiFi en remplaçant un compte local unique par un annuaire centralisé. Elle permet également de centraliser la gestion des utilisateurs et de différencier les niveaux d'accès entre les membres de l'équipe à mesure que le nombre d'utilisateurs augmente, plutôt que de partager un unique compte local. Les communications avec l'interface NiFi restent chiffrées en HTTPS.

Chapitre 4 — Réalisation technique du pipeline d'ingestion

4.1 Ingestion de fichiers plats

Le flux d'ingestion de fichiers plats prend en charge les formats CSV, JSON, XML et TXT délimité, chacun disposant de son propre Process Group de traitement. Les fichiers sont déposés dans un répertoire local dédié (data/input) plutôt que via un protocole de dépôt distant type SFTP : ce mode de dépôt, prévu par le cahier des charges, n'est pas implémenté pendant la durée du stage, mais resterait facilement intégrable dans l'architecture actuelle — il suffirait d'ajouter un processeur ListSFTP/FetchSFTP en amont du routage par extension déjà en place, sans modifier le reste du flux. Le flux détecte l'arrivée d'un fichier puis le route par extension (RouteOnAttribute) vers le Process Group correspondant au format détecté.

À l'intérieur de chaque Process Group de format, les données sont lues via l'API Record de NiFi (couple Record Reader / Record Writer propre à chaque format : CSVReader, JsonTreeReader...), puis converties vers le format CSV pivot via ConvertRecord, avant transmission au port de sortie commun vers le bloc Storage.

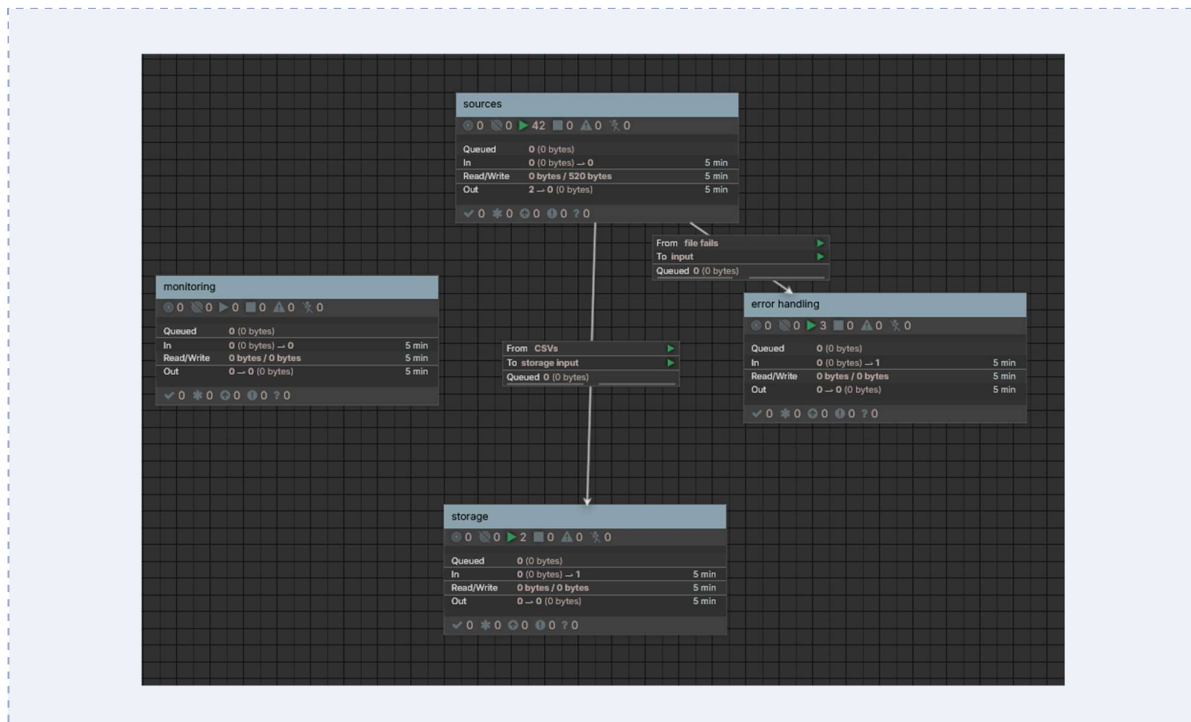


Figure 3. Vue des Process Groups principaux dans l'interface NiFi.

4.2 Ingestion JDBC

L'ingestion de base relationnelle s'appuie sur un Controller Service DBCPConnectionPool, configuré pour se connecter à une base MySQL via le pilote MySQL Connector/J — seul moteur de base de données effectivement intégré et testé pendant le stage, les connecteurs Oracle, SQL Server et PostgreSQL prévus par le cahier des charges restant une piste d'évolution (cf. chapitre 7.2).

L'extraction repose sur une logique incrémentale : une colonne de suivi (valeur maximale déjà extraite) permet de ne récupérer que les nouvelles données lors des exécutions suivantes, plutôt que de réextraire l'intégralité de la table à chaque run. L'architecture, construite autour de l'API JDBC standard, resterait compatible avec d'autres moteurs relationnels moyennant le remplacement du pilote JDBC et un test de validation, sans modification du flux NiFi lui-même.

4.3 Unification au format CSV

Quelle que soit la source d'origine (fichier ou base de données), les données sont systématiquement converties vers un format CSV unique avant d'atteindre le bloc de stockage. Ce choix d'un format pivot simplifie l'écriture HDFS et la création de la table Hive, qui n'ont besoin de gérer qu'un seul schéma de sérialisation en sortie de l'ingestion, quelle que soit la diversité des formats en entrée.

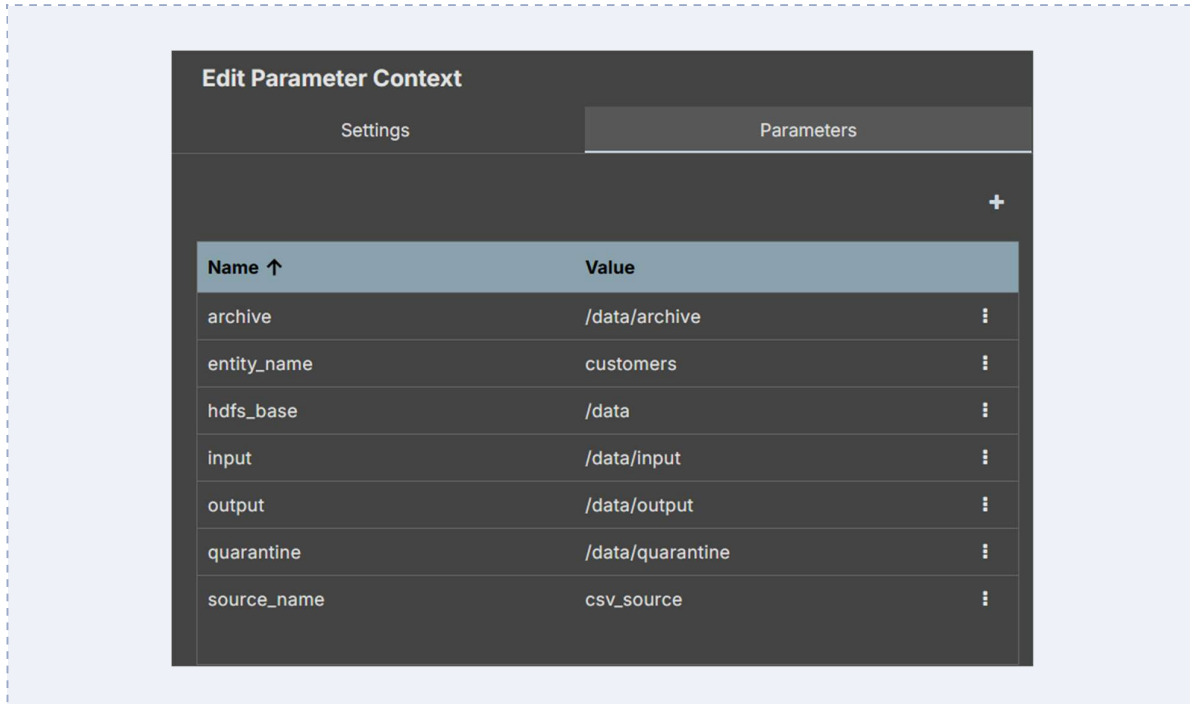


Figure 4. Paramétrage générique d'un Process Group via un Parameter Context.

4.4 Gestion des erreurs

Chaque Process Group de traitement de format intègre un mécanisme de retry local : en cas d'échec de conversion (ConvertRecord), le FlowFile est renvoyé en boucle vers le traitement via RetryFlowFile, avec un compteur de tentatives. Au-delà du nombre maximal de tentatives, le FlowFile est routé vers une zone de quarantaine dédiée, avec conservation des métadonnées d'erreur.

Un Process Group Error Handling centralise trois catégories d'échecs, reprises telles que nommées dans le flux : Failed Fetch (échec de récupération de fichier), Retry Exceeded (dépassement du nombre de tentatives) et Unsupported Files (aucun Process Group de traitement disponible pour l'extension détectée). Cette centralisation facilite l'ajout futur d'un traitement uniforme des erreurs (alerting, journalisation).

4.5 Stockage HDFS partitionné et Hive

L'écriture finale sur HDFS est réalisée via le processeur PutHDFS, avec un chemin de destination construit dynamiquement selon la convention source/entité/année/mois/jour décrite au chapitre précédent. Une table Hive est créée en cohérence avec cette arborescence, afin de permettre l'interrogation des données stockées via des requêtes SQL (notamment via le client beeline).

Le métastore Hive s'appuie sur la même instance MySQL que celle utilisée pour l'ingestion JDBC, dans une base dédiée (hive), distincte de la base source (ingestion). Cette mutualisation évite de déployer un service de base de données supplémentaire tout en conservant une séparation logique claire entre les métadonnées Hive et les données métier.

The screenshot shows the Hadoop Browse Directory interface. At the top, there's a navigation bar with links: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The main heading is "Browse Directory". Below it, a search bar contains the path "/data/csv_source/customers/year=2026/month=07/". To the right of the search bar are icons for file operations. Below the search bar, there's a "Show" dropdown set to "25" and a "Search:" input field. A table lists the directory entries with columns: Permission, Owner, Group, Size, Last Modified, Replication, Block Size, and Name. Two entries are shown, both with permissions "drwxr-xr-x", owner "nifi", group "supergroup", size "0 B", and last modified "Aug 02 10:47" and "Aug 02 10:49". The names are "day=16" and "day=19". Below the table, it says "Showing 1 to 2 of 2 entries". At the bottom, there's a "Previous" button, a "1" button, and a "Next" button. Below the directory listing, there's a section titled "Hadoop, 2019." followed by an "INFO" message: "Completed executing command(queryId=hive_20260802181529_9e3192bc-bac4-4477-8c14-39f843ecc3b7); Time taken: 0.0 seconds". Below this is a table with 8 columns: customers.id, customers.first_name, customers.last_name, customers.department, customers.salary, customers.year, customers.month, and customers.day. The table contains 7 rows of data. At the bottom, it says "7 rows selected (3.431 seconds)" and "0: jdbc:hive2://localhost:10000> |".

Figure 5. Arborescence HDFS partitionnée et table Hive correspondante.

4.6 Orchestration et supervision de bout en bout

Le DAG Airflow final_ingestion_pipeline orchestre l'exécution complète du pipeline : vérification et démarrage des conteneurs nécessaires, attente de la disponibilité des services, authentification à NiFi, déclenchement du Process Group d'ingestion, puis envoi d'une notification Discord récapitulant le résultat de l'exécution (succès/échec) et sa durée.

The screenshot shows the Airflow DAGs interface. At the top, there's a navigation bar with links: DAGs, Active, Paused, Running, and Failed. Below the navigation bar, there's a "Filter DAGs by tag" input field and a "Search DAGs" input field. To the right of the search bar is an "Auto-refresh" button. Below the search bar, there's a table with columns: DAG, Owner, Runs, Schedule, Last Run, Next Run, Recent Tasks, and Actions. Two DAGs are listed: "final_ingestion_pipeline" and "ingestion_pipeline". The "final_ingestion_pipeline" DAG is shown with a "Runs" column containing a green circle with a checkmark, indicating it is running. Below the DAGs table, there's a Discord notification window titled "pipeline notifs APP 7:21 PM". The notification content is: "Pipeline Succeeded ✓", "Runtime 0s", "Containers", a list of containers (mysql, namenode, datanode, ldap, phpldapadmin, phpmyadmin, hive, nifi) all running, and "NiFi Process Group: 6bf95b17-819f-1000-de47-473494322749".

Figure 6. Exécution du DAG Airflow et notifications Discord associées.

4.7 Organisation du dépôt de code

Le projet est versionné sur GitHub selon une structure modulaire par domaine technique (airflow/, control_center/, nifi/, hadoop/, hive/, data/), accompagnée d'un README détaillant l'architecture, les technologies utilisées et la procédure de démarrage. Le déploiement complet de la plateforme se résume à deux commandes :

```
git clone https://github.com/sunyata0-0/BigData-Ingestion-Pipeline.git  
docker compose up -d
```

Chapitre 5 — Le centre de contrôle Flask

Au terme du développement du pipeline d'ingestion, l'exploitation quotidienne de la plateforme nécessite de naviguer entre plusieurs interfaces distinctes : NiFi, Airflow, phpMyAdmin, le navigateur HDFS, la ligne de commande Docker et phpLDAPadmin. Afin de simplifier cette exploitation, un centre de contrôle web a été développé en fin de stage, au-delà du périmètre initialement fixé par le cahier des charges.

5.1 Objectif

Le centre de contrôle Flask centralise l'administration et la supervision de l'ensemble de la plateforme dans une interface web unique, accessible sur <http://localhost:5000>, démarrée indépendamment de la stack Docker via la commande `python app.py`. L'objectif n'est pas de remplacer les interfaces natives de chaque outil, mais d'offrir un point d'entrée unique pour les opérations courantes.

5.2 Architecture applicative

L'application est construite avec Flask selon une architecture modulaire par domaine fonctionnel : un blueprint et un service dédiés à chaque intégration (Docker, Airflow, HDFS, Hive, LDAP, MySQL, Discord, README/documentation, upload de fichiers). Cette séparation reprend, côté centre de contrôle, la même logique de modularité que celle appliquée aux flux NiFi et aux utilitaires Airflow.

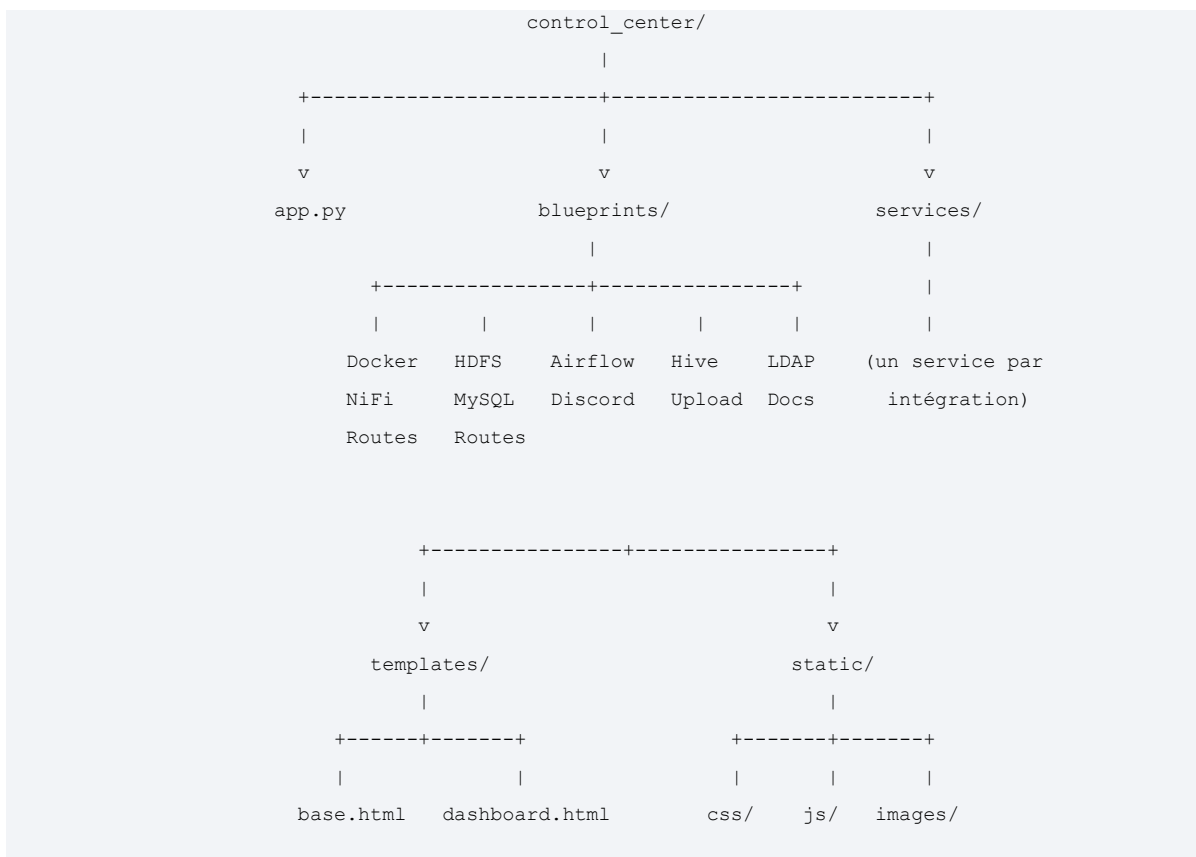


Figure 7. Structure applicative du centre de contrôle Flask.

Le frontend s'organise autour d'un système de cartes (une par service) donnant accès à des fenêtres modales réutilisables, avec une base CSS/JavaScript commune plutôt que du code dupliqué par module. La plupart des cartes ouvrent une modale dédiée permettant d'effectuer une action sans quitter le tableau de bord.

5.3 Fonctionnalités par module

Tableau 3. Fonctionnalités du centre de contrôle par module.

Module	Fonctionnalités
Docker	Statut des conteneurs, démarrage/arrêt individuel des services
Airflow	Upload de fichiers vers le dossier d'ingestion, déclenchement du DAG, retour d'exécution
HDFS	Navigation dans l'arborescence, consultation des répertoires et fichiers stockés
Hive	Liste des bases et tables, consultation du schéma, aperçu des 20 premières lignes
LDAP	Parcours des utilisateurs de l'annuaire, révélation d'identifiants après vérification administrateur
MySQL	Insertion de nouvelles lignes dans la base source (simulateur d'arrivée de données)
Discord	Envoi de notifications personnalisées vers le webhook du projet
NiFi	Visualisation interactive (SVG, zoom et déplacement) de l'architecture complète du pipeline
Documentation	Consultation du README et de la documentation du projet depuis le tableau de bord

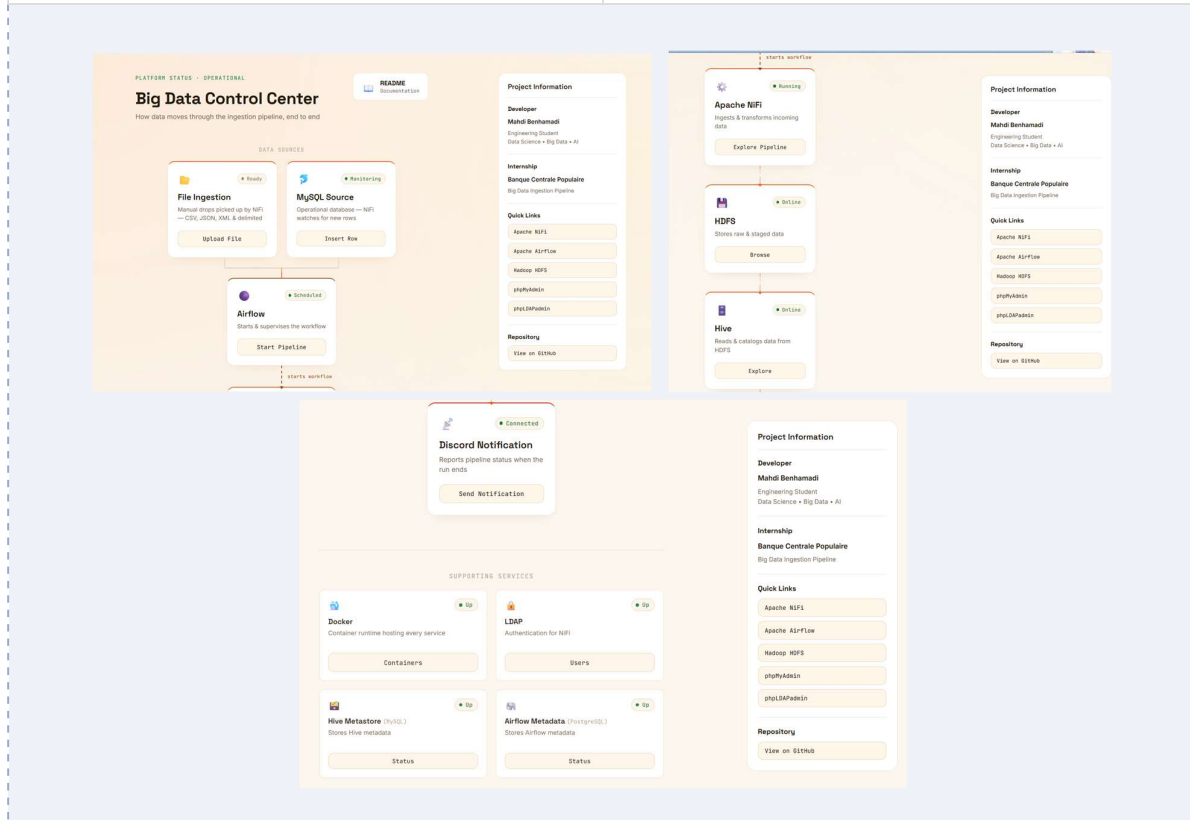


Figure 8. Tableau de bord principal du centre de contrôle.

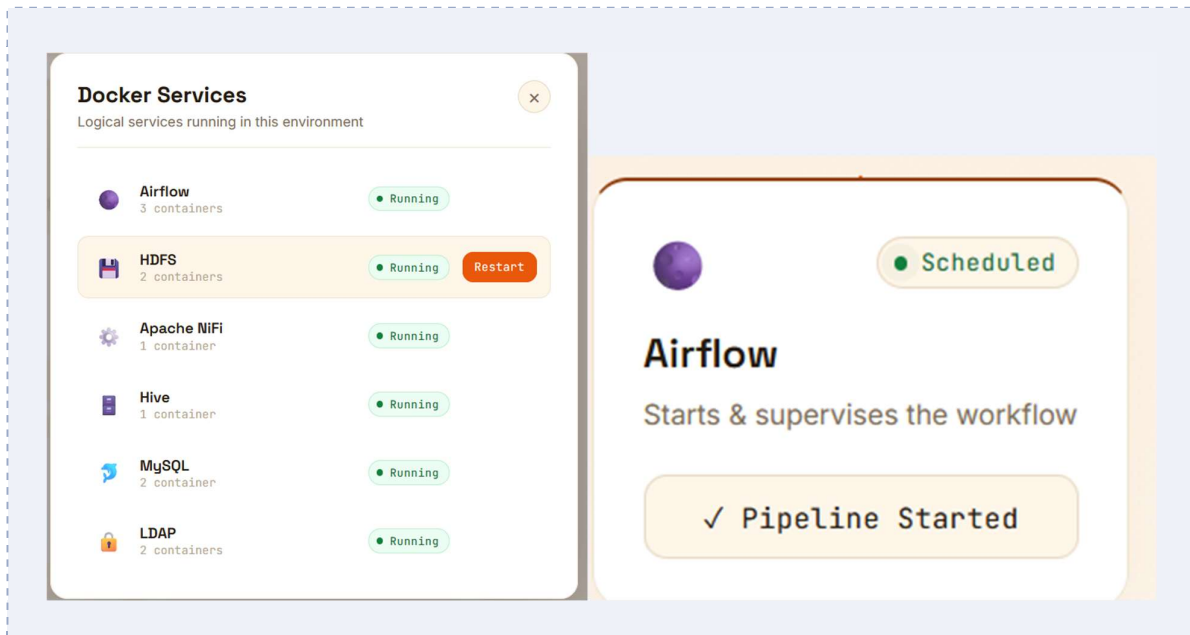


Figure 9. Modules Docker et Airflow du centre de contrôle.

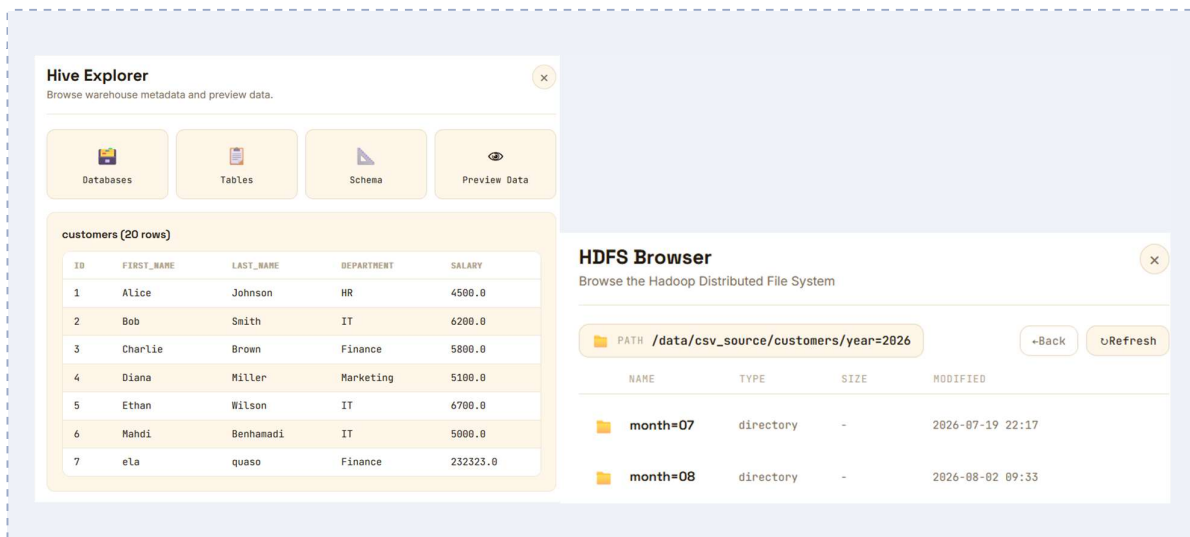


Figure 10. Modules Hive et HDFS du centre de contrôle.

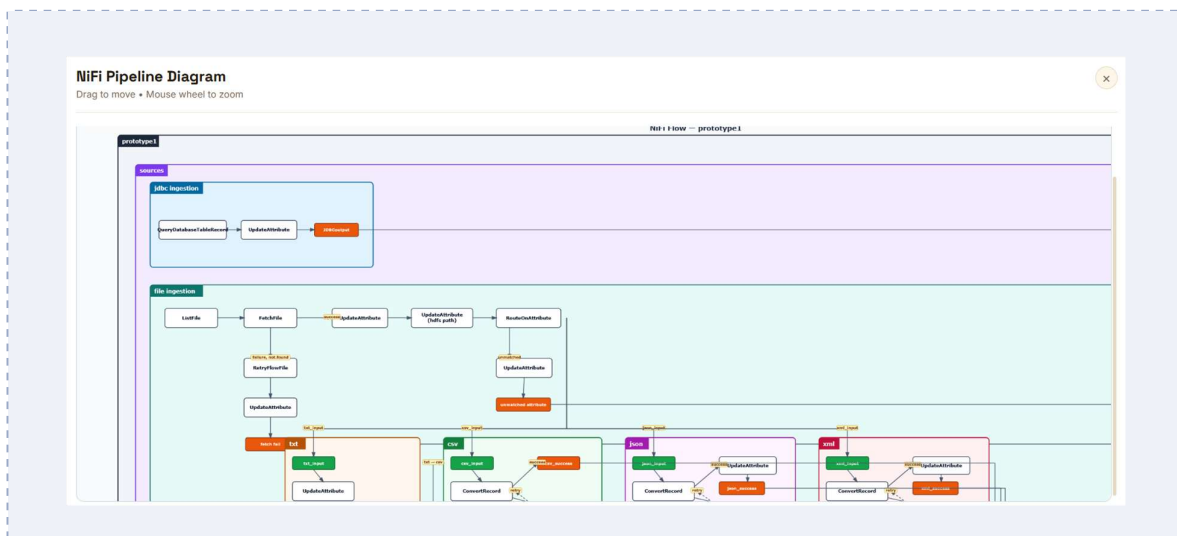


Figure 11. Visualisation interactive du pipeline NiFi dans le centre de contrôle.

21

Chapitre 6 — Difficultés rencontrées et démarche de résolution

La réalisation du projet s'est accompagnée de plusieurs difficultés techniques, inhérentes à la découverte simultanée de plusieurs technologies. Le tableau ci-dessous synthétise les principales difficultés rencontrées et la manière dont elles ont été résolues.

Tableau 4. Difficultés rencontrées et résolutions apportées.

Difficulté rencontrée	Résolution apportée
Ressources limitées sur la machine de développement (disque presque plein, 16 Go de RAM)	Choix d'une stack Docker Compose légère plutôt qu'une machine virtuelle Cloudera complète
Renommage de fichiers conservant l'ancienne extension (\${filename}.txt)	Utilisation d'une expression NiFi avec remplacement par regex (replace()) pour cibler l'extension existante
Confusion entre validation de type de fichier et validation de schéma	Séparation claire : RouteOnAttribute pour le routage par format, ValidateRecord pour la validation de structure
Processeurs HDFS (PutHDFS) absents de l'installation NiFi de base	Ajout des NAR Hadoop, puis gestion des extensions via le volume Docker de NiFi
Attributs de partitionnement vides (\${ingestion.year})	Correction du nommage des attributs NiFi et séparation des étapes de création de métadonnées et de génération de chemin
Perte des utilisateurs LDAP après redémarrage des conteneurs	Ajout de volumes Docker persistants pour les données et la configuration LDAP
Échec de l'API Airflow → NiFi après migration vers LDAP	Mise à jour de l'authentification utilisée par le DAG Airflow pour s'aligner sur le nouveau mode LDAP
Structure de réponse de l'API NiFi différente de celle attendue (KeyError)	Inspection directe de la réponse JSON et adaptation du parsing aux Process Groups enfants
Connexion Git associée au mauvais compte GitHub	Nettoyage des identifiants stockés via l'utilitaire Windows cmdkey

Au-delà de ces points techniques ponctuels, la principale difficulté du stage réside dans la nécessité d'apprendre un écosystème Big Data complet (NiFi, Hadoop, Hive, Airflow, LDAP, Flask) en un temps réduit, tout en produisant simultanément une solution fonctionnelle. Cette contrainte oriente la démarche vers un apprentissage ciblé : plutôt que d'étudier chaque outil de façon exhaustive, chaque notion est apprise au moment précis où elle devient nécessaire à l'avancement du flux ou du module en cours, ce qui permet de rester concentré sur les exigences réelles du projet plutôt que sur l'exhaustivité théorique.

Chapitre 7 — Résultats et bilan

7.1 Livrables réalisés

Tableau 5. Livrables réalisés au regard du cahier des charges.

Livrable attendu	État de réalisation
Étude de cadrage	Analyse de l'existant et spécifications réalisées, adaptées après le passage de Cloudera à Docker
Flux NiFi opérationnels	Process Groups paramétrables pour fichiers plats (CSV/JSON/XML/TXT) et ingestion JDBC MySQL
Module de gestion des erreurs	Retry paramétrable et quarantaine, centralisés dans un Process Group Error Handling
Traçabilité des exécutions	Notifications d'exécution via Discord (succès/échec, durée) ; historisation possible via les logs NiFi/Airflow
Zone de stockage HDFS	Arborescence partitionnée par date opérationnelle, avec une table Hive alignée
Documentation technique	README détaillé (architecture, technologies, procédure de démarrage) publié sur GitHub
Centre de contrôle (hors périmètre initial)	Tableau de bord Flask opérationnel, couvrant Docker, Airflow, HDFS, Hive, LDAP, MySQL, Discord et la visualisation NiFi
Rapport de stage	Présent document

7.2 Écarts par rapport au cahier des charges

Certains éléments du périmètre initial n'ont pas été couverts pendant la durée du stage et constituent des pistes d'évolution plutôt que des fonctionnalités finalisées :

- le dépôt de fichiers se fait via un répertoire local (data/input) et non via SFTP ;
- le format à largeur fixe n'est pas implémenté, seuls CSV/JSON/XML/TXT le sont ;
- seul MySQL est intégré et testé pour l'ingestion JDBC, à l'exclusion d'Oracle, SQL Server et PostgreSQL ;
- l'architecture Medallion (Bronze/Silver/Gold) évoquée en contexte n'est pas mise en œuvre ;
- le Process Group Monitoring existe dans le flux mais n'est pas encore actif ;
- l'alerting est assuré via Discord plutôt que par email/Teams comme suggéré initialement ;
- Le format de stockage retenu est CSV, avec une évolution prévue vers un format optimisé (Parquet/Avro).

7.3 Perspectives d'amélioration

- Intégration des connecteurs Oracle, SQL Server et PostgreSQL pour l'ingestion JDBC ;
- Mise en place d'un Process Group Monitoring réellement actif, avec indicateurs de supervision ;
- Migration du format de stockage pivot vers Parquet ou Avro ;
- Mise en œuvre d'une architecture Medallion complète (Bronze/Silver/Gold) ;
- Authentification du centre de contrôle lui-même (actuellement accessible sans connexion dédiée) ;
- Automatisation de la création des tables Hive lors de l'ajout d'une nouvelle source ;
- Mise en place de contrôles de qualité de données plus poussés en amont de l'ingestion.

7.4 Bilan personnel

Ce stage constitue une première immersion complète dans un écosystème Big Data de bout en bout, depuis l'ingestion jusqu'au stockage, en passant par l'orchestration, la sécurisation des accès et, in fine, le développement d'une interface d'administration complète. Il permet de consolider des compétences en Apache NiFi, conteneurisation Docker, orchestration Airflow, développement web Flask, ainsi qu'en résolution autonome de problèmes techniques dans un environnement encore peu maîtrisé au démarrage. La nécessité de s'adapter à une contrainte de ressources en cours de route (Cloudera → Docker), puis de dépasser le périmètre initial avec le centre de contrôle, constituent des expériences formatrices en matière d'adaptabilité technique et de prise d'initiative.

Conclusion générale

Ce stage d'initiation, organisé sous la forme d'un Projet de Fin d'Année, permet de concevoir et développer une plateforme d'ingestion de données automatisée et réutilisable, couvrant une partie du périmètre fixé par le cahier des charges de la Banque Centrale Populaire, puis de l'étendre avec un centre de contrôle Flask développé de sa propre initiative. Face à des contraintes de ressources sur la machine de développement, l'environnement Cloudera CDP initialement prévu est remplacé par une stack Docker orchestrée par Airflow, aboutissant à une solution modulaire couvrant l'ingestion de fichiers plats et JDBC (MySQL), la gestion des erreurs, le stockage partitionné sur HDFS, et une administration centralisée de l'ensemble de la plateforme.

Au-delà du livrable technique, ce stage représente un parcours d'apprentissage intensif sur l'écosystème Big Data, mené sans expérience préalable sur la majorité des technologies utilisées. Les compétences acquises en NiFi, Hadoop, Hive, Airflow, Docker et développement Flask, ainsi que la méthodologie de résolution de problèmes développée au fil du projet, constituent une base solide pour la poursuite d'un parcours en science des données et Big Data.

Bibliographie

Les ressources suivantes, consultées tout au long du stage, ont servi de référence pour la conception et le développement de la plateforme :

- Apache NiFi — Documentation officielle, <https://nifi.apache.org/documentation/>
- Apache Airflow — Documentation officielle, <https://airflow.apache.org/docs/>
- Apache Hadoop — Documentation officielle, <https://hadoop.apache.org/docs/stable/>
- Apache Hive — Documentation officielle, <https://hive.apache.org/>
- Docker — Documentation officielle, <https://docs.docker.com/>
- Flask — Documentation officielle, <https://flask.palletsprojects.com/>
- OpenLDAP — Documentation officielle, <https://www.openldap.org/doc/>
- Dépôt du projet — github.com/sunyata0-0/BigData-Ingestion-Pipeline
- Exactement 100 vidéos sur Youtube

Annexes

Annexe A — Structure du dépôt de code

```
bigdata-ingestion-pipeline/
|
├─ airflow/
|   └─ dags/
|       └─ final_ingestion_pipeline.py
|       └─ utils/
|
├─ control_center/
|   └─ app.py
|   └─ blueprints/
|   └─ services/
|   └─ templates/
|   └─ static/
|
├─ data/
|   └─ input/
|   └─ output/
|   └─ quarantine/
|   └─ archive/
|
├─ hadoop/
├─ hive/
├─ nifi/
|
├─ docker-compose.yml
└─ README.md
```

Annexe B — Services déployés via Docker Compose

- Apache NiFi — moteur d'ingestion (<https://localhost:8444>)
- Apache Airflow — orchestration (<http://localhost:8082>)
- MySQL — source JDBC et métastore Hive (port hôte 3307)
- phpMyAdmin — administration MySQL (<http://localhost:8081>)
- Hadoop NameNode / DataNode — stockage HDFS (<http://localhost:9870>)
- Apache Hive — couche table SQL sur HDFS (ports 10000/10002)
- OpenLDAP — annuaire d'authentification (port 389)
- phpLDAPadmin — administration LDAP (<https://localhost:8085>)
- PostgreSQL — base de métadonnées Airflow
- Centre de contrôle Flask — <http://localhost:5000> (lancé séparément via python app.py)

Annexe C — Lien du dépôt

Code source complet disponible sur GitHub : github.com/sunyata0-0/BigData-Ingestion-Pipeline

Table des matières

Remerciements

Résumé

Introduction générale

Chapitre 1 — Contexte et présentation du projet

- 1.1 Organisme d'accueil
- 1.2 Contexte et problématique
- 1.3 Objectifs du projet
- 1.4 Périmètre visé par le cahier des charges

Chapitre 2 — Démarche d'apprentissage et choix technologiques

- 2.1 Un écosystème découvert en cours de stage
- 2.2 Du Cloudera CDP à une stack conteneurisée
- 2.3 Technologies retenues

Chapitre 3 — Conception de l'architecture

- 3.1 Vue d'ensemble
- 3.2 Architecture des flux NiFi
- 3.3 Généricité : Parameter Contexts et Variables Airflow
- 3.4 Convention de partitionnement HDFS
- 3.5 Orchestration Airflow
- 3.6 Sécurité et authentification

Chapitre 4 — Réalisation technique du pipeline d'ingestion

- 4.1 Ingestion de fichiers plats
- 4.2 Ingestion JDBC
- 4.3 Unification au format CSV
- 4.4 Gestion des erreurs
- 4.5 Stockage HDFS partitionné et Hive
- 4.6 Orchestration et supervision de bout en bout
- 4.7 Organisation du dépôt de code

Chapitre 5 — Le centre de contrôle Flask

- 5.1 Objectif
- 5.2 Architecture applicative
- 5.3 Fonctionnalités par module
- 5.4 Apport du centre de contrôle

Chapitre 6 — Difficultés rencontrées et démarche de résolution

Chapitre 7 — Résultats et bilan

- 7.1 Livrables réalisés
- 7.2 Écarts par rapport au cahier des charges
- 7.3 Perspectives d'amélioration
- 7.4 Bilan personnel

Conclusion générale

Bibliographie

Annexes